

**LAPORAN RINGKAS EKSPERIMEN MIKROKONTROLLER (ESP32 /
iMCLab) – Poin 1**

Dosen Pengampu:

Dr. Basuki Rahmat, S.Si. MT



Disusun oleh:

Riky Hermawan

23081010021

**MIKROKONTROLLER A – 081
PROGRAM STUDI INFORMATIKA
FAKULTAS ILMU KOMPUTER
UNIVERSITAS PEMBANGUNAN NASIONAL “ VETERAN”
JAWA TIMUR
2025**

PROGRAM 1

ESP32 IoT DC Motor Control: MQTT ON/OFF + PWM Speed Control

Kode Program:

```
#include <WiFi.h>
#include <PubSubClient.h>
#include <Arduino.h>

int ledPin = 2;
char myData = 0;

int motor1Pin1 = 27;
int motor1Pin2 = 26;
int enable1Pin = 12;

// ===== PWM Motor =====
const int freq = 30000;
const int pwmChannel = 0;
const int resolution = 8; // 0..255
int dutyCycle = 200;

// ===== PWM LED (ikut speed) =====
const int ledChannel = 1; // channel berbeda dari motor
const int ledResolution = 8; // 0..255

// ===== RPM (belum dipakai) =====
const byte pin_rpm = 13;
int volatile rev = 0;

// ===== WiFi =====
const char* ssid = "Griya Palem lantai 1";
const char* password = "13456789";

// ===== MQTT =====
#define mqttServer "broker.emqx.io"
#define mqttPort 1883

WiFiServer server(80);
WiFiClient espClient;
PubSubClient client(espClient);

String Topic;
String Payload;

// constants
const int baud = 115200;

// ===== Helper: LED mengikuti speed =====
void setLedBySpeed(int speedValue) {
```

```

// speedValue 0..255
if (speedValue < 0) speedValue = 0;
if (speedValue > 255) speedValue = 255;

// versi langsung (LED = speed)
ledcWrite(ledChannel, speedValue);

// Kalau LED terlalu redup saat speed kecil, pakai mapping ini (optional):
// if (speedValue == 0) ledcWrite(ledChannel, 0);
// else ledcWrite(ledChannel, map(speedValue, 1, 255, 30, 255));
}

void MotorOn()
{
  // Motor forward speed default
  dutyCycle = 205;
  digitalWrite(motor1Pin1, HIGH);
  digitalWrite(motor1Pin2, LOW);
  ledcWrite(pwmChannel, dutyCycle);

  // LED ikut speed motor
  setLedBySpeed(dutyCycle);
}

void MotorOff()
{
  dutyCycle = 0;
  // stop PWM
  ledcWrite(pwmChannel, dutyCycle);

  // optional: lepas arah (coast)
  // digitalWrite(motor1Pin1, LOW);
  // digitalWrite(motor1Pin2, LOW);

  // LED ikut mati
  setLedBySpeed(0);
}

void receivedCallback(char* topic, byte* payload, unsigned int length) {

  // ===== ON/OFF (payload '1' / '0') =====
  if ((char)payload[0] == '1') {
    MotorOn();
    Serial.println("Motor On");
  }

  if ((char)payload[0] == '0') {
    MotorOff();
    Serial.println("Motor Off");
  }
}

```

```

// ===== kontrol kecepatan via topic kontrolspeed (payload angka 0..255) =====
if (String(topic) == "kontrolspeed") {
    String msg = "";
    for (int i = 0; i < (int)length; i++) {
        msg += (char)payload[i];
    }
    msg.trim();

    int speedValue = msg.toInt();
    if (speedValue < 0) speedValue = 0;
    if (speedValue > 255) speedValue = 255;

    // pastikan arah tetap maju
    digitalWrite(motor1Pin1, HIGH);
    digitalWrite(motor1Pin2, LOW);

    // set PWM motor
    dutyCycle = speedValue;
    ledcWrite(pwmChannel, dutyCycle);

    // LED ikut speed motor
    setLedBySpeed(dutyCycle);

    Serial.print("Kecepatan diubah ke: ");
    Serial.println(dutyCycle);
}
}

void setup()
{
    Serial.begin(115200);

    // pins
    pinMode(motor1Pin1, OUTPUT);
    pinMode(motor1Pin2, OUTPUT);
    pinMode(enable1Pin, OUTPUT);

    pinMode(pin_rpm, INPUT_PULLUP);

    // ===== PWM setup motor =====
    ledcSetup(pwmChannel, freq, resolution);
    ledcAttachPin(enable1Pin, pwmChannel);
    ledcWrite(pwmChannel, 0);

    // ===== PWM setup LED =====
    ledcSetup(ledChannel, freq, ledResolution);
    ledcAttachPin(ledPin, ledChannel);
    ledcWrite(ledChannel, 0);
}

```

```

Serial.println("Testing DC Motor...");

// ===== WiFi connect =====
Serial.println();
Serial.print("Connecting to ");
Serial.println(ssid);
WiFi.begin(ssid, password);

while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
}
Serial.println();
Serial.println("WiFi connected");

// ===== MQTT connect =====
client.setServer(mqttServer, mqttPort);
client.setCallback(receivedCallback);

while (!client.connected()) {
    Serial.println("Connecting to Cloud IoT ...");

    if (client.connect("ESP32_Riky")) {
        Serial.println("connected");
    } else {
        Serial.print("failed with state ");
        Serial.println(client.state());
        delay(2000);
    }
}

// subscribe setelah connect (lebih aman)
client.subscribe("kontrolonoff");
client.subscribe("kontrolspeed");
}

void loop()
{
    client.loop();

    // kontrol via Serial Monitor (kirim '1' atau '0')
    myData = (char)Serial.read();

    if (myData == '1') {
        MotorOn();
        Serial.println("Motor+LED ON (Serial)");
    }
    else if (myData == '0') {
        MotorOff();
        Serial.println("Motor+LED OFF (Serial)");
    }
}

```

```
}  
}
```

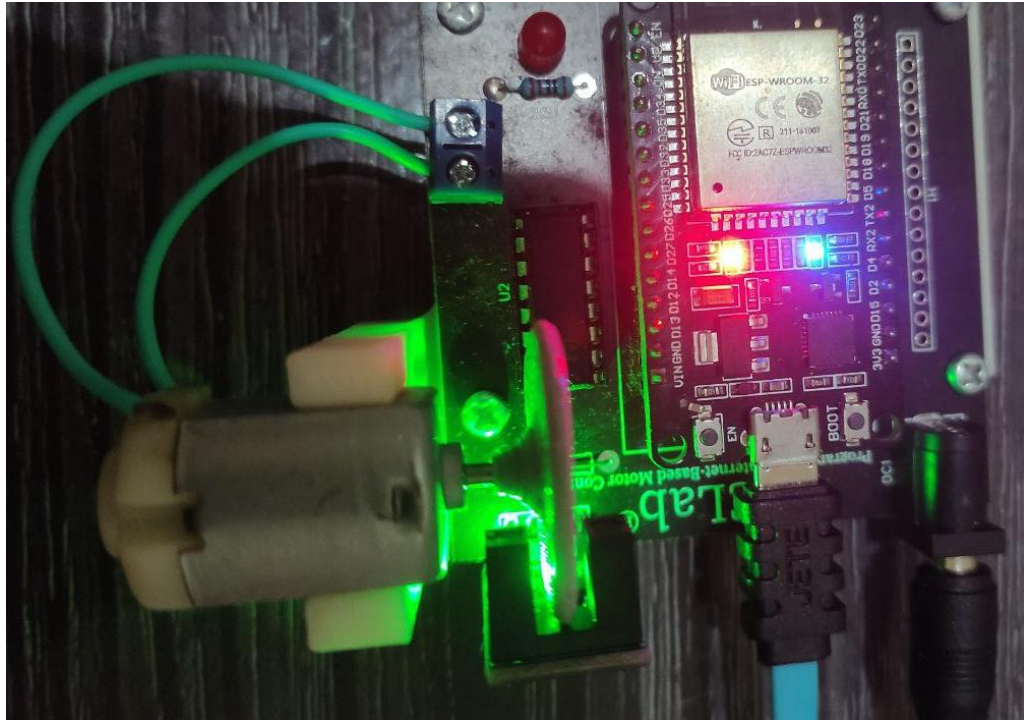
Hasil Program:

Sistem berhasil:

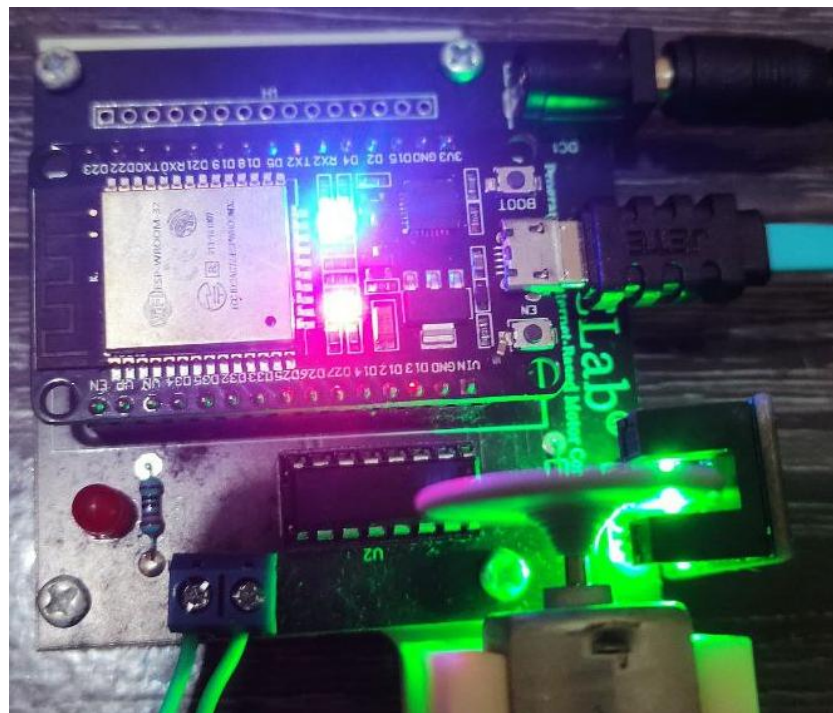
1. Terkoneksi WiFi dan menampilkan status pada Serial Monitor.
2. Terkoneksi MQTT broker dan melakukan subscribe ke 2 topik:
 - kontrolonoff (payload: 1 / 0)
 - kontrolspeed (payload: angka 0..255)
3. Motor DC dapat:
 - ON/OFF via MQTT (kontrolonoff)
 - diatur kecepatannya via MQTT (kontrolspeed)
 - dikendalikan manual via Serial Monitor (1/0) jika MQTT tidak digunakan.

```
WiFi connected  
Connecting to Cloud IoT ...  
connected  
Motor+LED ON (Serial)  
Motor+LED OFF (Serial)  
Motor On  
Motor On  
Kecepatan diubah ke: 145  
Motor On  
Kecepatan diubah ke: 185  
Kecepatan diubah ke: 205  
Motor On  
Kecepatan diubah ke: 180  
Motor On  
Kecepatan diubah ke: 165  
Motor On  
Kecepatan diubah ke: 140  
Motor On  
Kecepatan diubah ke: 120
```

Gambar 1. Tampilan serial monitor



Gambar 2. Tampilan led saat speed rendah



Gambar 3. Tampilan led saat speed tinggi

Analisis:

Eksperimen ini menggabungkan kontrol IoT dan kontrol lokal. MQTT berperan sebagai jalur kendali jarak jauh: topik kontrolonoff untuk logika saklar, dan kontrolspeed untuk nilai PWM. PWM menggunakan resolusi 8-bit sehingga nilai duty berada pada rentang 0–255; semakin tinggi duty cycle, semakin besar rata-rata tegangan pada motor sehingga motor cenderung semakin cepat.

Temuan penting adalah motor tidak selalu berputar pada PWM rendah (misalnya di bawah kisaran tertentu). Hal ini wajar karena motor DC memerlukan torsi awal (starting torque) dan akan “kalah” oleh gesekan/ beban saat duty kecil. Akibatnya, walaupun PWM sudah dikirim, motor hanya berdengung atau diam. Solusi umumnya: menaikkan duty minimal (threshold), memakai driver yang lebih baik, supply yang stabil, atau teknik “kick-start” (duty tinggi sebentar lalu turun ke target).

PROGRAM 2

Python Serial Automation: LED Fade 100% → 0%

Kode Program:

```
import time
import serial

PORT = "COM3"
BAUD = 115200

ser = serial.Serial(PORT, BAUD, timeout=1)
time.sleep(2) # tunggu board reset

while ser.in_waiting:
    print(ser.readline().decode(errors="ignore").strip())

def send(cmd: str):
    ser.write((cmd.strip() + "\n").encode())

def read_lines(seconds=0.2):
    time.sleep(seconds)
    out = []
    while ser.in_waiting:
        out.append(ser.readline().decode(errors="ignore").strip())
    return out

# LED ON
send("LED 1")
print("\n".join(read_lines()))

# Fade 100 -> 0
for p in range(100, -1, -10):
    send(f"POWER {p}")
    lines = read_lines(0.15)
    if lines:
        print(lines[-1])

# LED OFF
send("LED 0")
print("\n".join(read_lines()))
```

```
ser.close()
print("Disconnected")
```

Hasil Program:

LED dapat berubah terang ke redup secara bertahap (fade) dari 100% sampai 0% menggunakan perintah serial yang dikirim otomatis dari Python.

```
READY
LED on
LED Power 100
LED Power 90
LED Power 80
LED Power 70
LED Power 60
LED Power 50
LED Power 40
LED Power 30
LED Power 20
LED Power 10
LED Power 0
LED off
Disconnected
```

Gambar 3. Hasil fade LED

Analisis:

Python berfungsi sebagai “controller” yang mengirim string perintah lewat serial: LED 1, POWER n, dan LED 0. Dengan pendekatan ini, proses pengujian menjadi cepat karena tidak perlu upload firmware berulang untuk tiap variasi nilai. Namun, eksperimen ini mensyaratkan firmware ESP32/Arduino yang memang mem-parsing perintah teks LED dan POWER. Jika firmware yang aktif hanya membaca karakter 1/0, maka perintah LED 1 dan POWER 50 tidak akan dikenali. Jadi, keberhasilan eksperimen ini bergantung pada kesesuaian protokol serial antara Python dan program mikrokontroler.